

School of Advanced Computing & Information Technology

Department of Computer Science, Gujarat University

SEC-316 Advanced Python Programming (Practical)

Assignment - Unit 1

- **High order functions: map, reduce and filter, Function Decorators, Generators**

1. Convert a list of integers into their squares using map().
2. Convert a list of temperatures in Celsius into Fahrenheit using map().
3. Convert all strings in a list to uppercase using map().
4. Given a list of numbers, use map() to find the cube of each number.
5. Use map() to calculate the length of each word in a list of words.
6. Given two lists, use map() to add the elements pairwise.
7. Convert a list of strings representing numbers into integers using map().
8. Use map() to round off a list of floating-point numbers to 2 decimal places.
9. Given a list of student marks (0–100), use map() to convert them into grades (A, B, C, D, F).
10. Use map() to create a list of factorial values for numbers from 1 to 10.
11. Use filter() to extract even numbers from a list.
12. Use filter() to extract numbers greater than 10 from a list.
13. Use filter() to extract words longer than 4 letters from a list of words.
14. From a list of numbers, use filter() to get all prime numbers.
15. Use filter() to extract only positive numbers from a list.
16. From a list of email addresses, use filter() to keep only Gmail addresses.
17. From a list of strings, use filter() to keep only palindromes.
18. From a list of tuples (name, age), use filter() to keep only people older than 18.
19. From a list of numbers, use filter() to keep only Armstrong numbers.
20. From a list of words, use filter() to select only words that start and end with the same letter.
21. Use reduce() to calculate the sum of all numbers in a list.
22. Use reduce() to calculate the product of all numbers in a list.

23. Use `reduce()` to find the maximum number in a list.
24. Use `reduce()` to find the minimum number in a list.
25. Use `reduce()` to concatenate all strings in a list into one string.
26. Use `reduce()` to find the Greatest Common Divisor (GCD) of a list of numbers.
27. Use `reduce()` to calculate the sum of squares of all numbers in a list.
28. Use `reduce()` to calculate the total salary from a list of dictionaries containing employee details.
29. Use `reduce()` to compute the running difference (subtract elements one by one).
30. Use `reduce()` to implement a custom sorting logic (e.g., longest string in a list).
31. A company stores employee salaries in thousands (e.g., $25 \rightarrow 25,000$). Use `map()` to convert them into actual salaries.
32. You have a list of product weights in kilograms. Convert them into grams using `map()`.
33. A list of student roll numbers is stored as strings. Convert them into integers using `map()`.
34. From a list of email addresses, filter only Gmail accounts.
35. From a dataset of employees (name, department), filter only employees from "HR".
36. From a list of job applicants (name, `experience_years`), filter those with ≥ 5 years of experience.
37. From a list of salaries, find the total payroll of a company.
38. From a list of words, join them to form a complete sentence.
39. From a list of exam scores, find the maximum score.
40. Write a generator to yield the first 10 natural numbers.
41. Create a generator that yields the squares of numbers from 1 to 10.
42. Build a generator to yield even numbers from a given range.
43. Create a generator that reads a list of student names and yields them one by one.
44. Write a generator to yield Fibonacci numbers up to `n`.
45. Create a generator that yields characters of a word one at a time (useful for text processing).
46. Build a generator that yields chunks of data (e.g., splitting a large dataset of 1000 entries into batches of 100).
47. Create a generator to simulate a traffic light system (Red $\rightarrow$ Green $\rightarrow$ Yellow $\rightarrow$ Red...).
48. Write a generator to yield daily stock prices from a list (to simulate streaming financial data).
49. Create a generator that yields log messages with timestamps (for system monitoring).
50. Write a decorator to print "Hello" before and "Goodbye" after a function runs.
51. Create a decorator to uppercase the output of a string-returning function.

52. Write a decorator that counts how many times a function is called.
53. Medium
54. Create a decorator to check if a user is authenticated before allowing access to a function (like login systems).
55. Write a decorator that logs function execution time (useful for performance testing).
56. Write a decorator to ensure a function always returns a positive number.
57. Create a decorator to restrict access based on user roles (e.g., admin vs normal user).
58. Write a decorator that automatically retries a function if it fails (e.g., API call).
59. Build a decorator for caching results of a function (useful for expensive calculations like factorial or Fibonacci).
60. Create a decorator to log function calls into a file with timestamp (for audit in computer systems).
61. Convert file sizes from KB to MB.
62. Find valid email IDs from a list.
63. Yield student roll numbers one by one.
64. Count how many times a function is executed.
65. Restrict access (only admin can run the function)

- **NumPy**

1. Write a NumPy program (using NumPy) to sum of all the multiples of 3 or 5 below 100.
2. Write a NumPy program to combine a one and a two-dimensional array together and display their elements.
3. Write a NumPy program to compute the multiplication of two given matrixes.
4. Write a NumPy program to compute the cross product of two given matrixes.
5. Write a NumPy program to compute the determinant of a given square array.
6. Write a NumPy program to compute the eigenvalues and right eigenvectors of a given square array.
7. Write a NumPy program to compute the inverse of a given matrix.
8. Write a NumPy program to compute the mean, standard deviation, and variance of a given array along the second axis.
9. Write a NumPy program to compute the covariance matrix of two given arrays.
10. Create a 1D NumPy array of numbers from 1 to 10.
11. Create a 1D NumPy array of 10 zeros.

12. Create a 1D NumPy array of 10 ones.
13. Create a 1D NumPy array of 10 fives.
14. Create a 1D array of numbers from 10 to 50.
15. Create a 1D array of even numbers from 10 to 50.
16. Create a 1D array of odd numbers from 1 to 20.
17. Create a 3x3 matrix with values ranging from 0 to 8.
18. Create a 3x3 identity matrix.
19. Create a 5x5 matrix with values 1,2,3,4 just below the diagonal.
20. Create an array of 100 equally spaced points between 0 and 1.
21. Create an array of 25 random numbers sampled from a standard normal distribution.
22. Create a 10x10 matrix with values from 0.01 to 1.00.
23. Create a 5x5 matrix with row values ranging from 0 to 4.
24. Create a 5x5 matrix with column values ranging from 0 to 4.
25. Create a 10x10 matrix with 1 on the border and 0 inside.
26. Create a 5x5 matrix with random values and find its maximum and minimum values.
27. Create a random vector of size 30 and find the mean value.
28. Create a 10x10 matrix with random values and normalize it.
29. Create a 5x5 matrix with random values and subtract the row mean.
30. Create a checkerboard 8x8 matrix using slicing.
31. Create a checkerboard 8x8 matrix using tile function.
32. Create a 10x3 matrix and sort each column.
33. Create a 10x3 matrix and sort each row.
34. Create a 5x5 matrix with random values and swap two rows.
35. Create a 5x5 matrix with random values and swap two columns.
36. Multiply a 5x3 matrix by a 3x2 matrix (real matrix product).
37. Create a 5x5 matrix with random integers and find the unique elements.
38. Create a 5x5 matrix and find the sum of diagonal elements.
39. Create a 4x4 matrix with values 0 to 15 and find the sum by rows and by columns.
40. Create a 5x5 matrix and find the sum of all elements.
41. Create a 5x5 matrix and find the mean of all elements.
42. Create a 5x5 matrix and find the standard deviation and variance of all elements.
43. Create a 5x5 matrix and find the minimum and maximum values along each row and column.
44. Create a 3x3 matrix with values from 1 to 9 and compute its determinant.

45. Create a 3x3 matrix with values from 1 to 9 and compute its inverse.

46. Solve the linear system of equations:

$$3x + y = 9,$$

$$x + 2y = 8.$$

47. Compute the eigenvalues and eigenvectors of a 2x2 matrix.

48. Compute the dot product of two arrays.

49. Compute the cross product of two vectors in 3D.

50. Create two random arrays and compute their correlation coefficient.

51. Create two random arrays and compute their covariance matrix.

52. Create a 5x5 matrix with random values and replace the maximum value with 0.

53. Create a 5x5 matrix with random values and replace the minimum value with 0.

54. Create a 5x5 matrix with random values and clip all values between 0.2 and 0.8.

55. Create a random 1D array of 20 elements and replace all values greater than 0.5 with 1.

56. Create a random 1D array of 20 elements and replace all values less than 0.5 with -1.

57. Create a 5x5 matrix with values 1 to 25 and reshape it into 25x1.

58. Create two 5x5 matrices and stack them horizontally and vertically.

59. Split a 1D array of 10 elements into 2 equal parts.

60. How to reshape an array?